

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №2
“Геометрические фигуры”

A handwritten signature in black ink, likely belonging to the student, is positioned above the student's name.

Выполнил:
Студент группы ИУ5-31Б
Никоноров Д.М.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Задание:

Разработать программу, реализующую работу с классами.

1. Программа должна быть разработана в виде консольного приложения на языке C#.
2. Абстрактный класс «Геометрическая фигура» содержит виртуальный метод для вычисления площади фигуры.
3. Класс «Прямоугольник» наследуется от «Геометрическая фигура». Ширина и высота объявляются как свойства (property). Класс должен содержать конструктор по параметрам «ширина» и «высота».
4. Класс «Квадрат» наследуется от «Прямоугольник». Класс должен содержать конструктор по длине стороны.
5. Класс «Круг» наследуется от «Геометрическая фигура». Радиус объявляется как свойство (property). Класс должен содержать конструктор по параметру «радиус».
6. Для классов «Прямоугольник», «Квадрат», «Круг» переопределить виртуальный метод `Object.ToString()`, который возвращает в виде строки основные параметры фигуры и ее площадь.
7. Разработать интерфейс `IPrint`. Интерфейс содержит метод `Print()`, который не принимает параметров и возвращает `void`. Для классов «Прямоугольник», «Квадрат», «Круг» реализовать наследование от интерфейса `IPrint`. Переопределяемый метод `Print()` выводит на консоль информацию, возвращаемую переопределенным методом `ToString()`.

Листинг кода:

Figure.cs

```
using System;
using System.Collections.Generic;
using System.Collections.Specialized;
using System.Runtime.CompilerServices;

namespace Figures
{
    abstract class Figure
    {
        public abstract double Area();
    };
} // Figures
```

IPrint.cs

```
namespace Figures
{
    interface IPrint
    {
        public void Print();
    };
}
```

```
}
```

Circle.cs

```
namespace Figures
```

```
{
```

```
    class Circle : Figure, IPrint
```

```
    {
```

```
        private double radius = 0;
```

```
        public double Radius
```

```
        {
```

```
            get { return this.radius; }
```

```
            set { this.radius = value; }
```

```
        }
```

```
        public Circle(double r = 0)
```

```
        {
```

```
            this.radius = r;
```

```
        }
```

```
        public override double Area()
```

```
        {
```

```
            return Math.PI * this.radius * this.radius;
```

```
        }
```

```
        public override string ToString()
```

```
        {
```

```
            return $"r = {this.radius}; area: {Math.Round(this.Area(), 2)}";
```

```
        }
```

```
        public void Print()
```

```
        {
```

```
            Console.WriteLine($"Kpyr: {this.ToString()}");
```

```
        }
```

```
    }
```

```
}
```

Rectangle.cs

```
namespace Figures
```

```
{
```

```
    class Rectangle : Figure, IPrint
```

```
    {
```

```
        private double width = 0;
```

```
        public double Width
```

```
        {
```

```
            get { return this.width; }
```

```
            set { width = value; }
```

```
        }
```

```
        private double height = 0;
```

```
        public double Height
```

```
        {
```

```
            get { return this.height; }
```

```

        set { this.height = value; }
    }

    public Rectangle(int w = 0, int h = 0)
    {
        this.width = w;
        this.height = h;
    }

    public override double Area()
    {
        return this.width * this.height;
        throw new NotImplementedException();
    }

    public override string ToString()
    {
        return $"w = {this.width}; h = {this.height}; area: {Math.Round(this.Area(), 2)}";
    }

    public void Print()
    {
        Console.WriteLine($"Прямоугольник: {this.ToString()}");
    }
}

```

Square.cs

namespace Figures

```

{
    class Square : Rectangle, IPrint
    {
        private double sideSize = 0;
        public double Side
        {
            get { return this.sideSize; }
            set { this.sideSize = value; }
        }

        public Square(int a = 0)
        {
            this.sideSize = a;
        }

        public override double Area()
        {
            return this.sideSize * this.sideSize;
        }

        public override string ToString()
        {
            return $"side = {this.sideSize}; area: {Math.Round(this.Area(), 2)}";
        }
    }
}

```

```

    }
}
}

```

Program.cs

```

using System;
using System.Collections.Generic;
using System.Diagnostics.Contracts;

namespace Figures
{
    class Program
    {
        static void Main(string[] argv)
        {
            // проверка работы конструкторов
            Rectangle rect1 = new Rectangle(10, 10);
            Square square1 = new Square(5);
            Circle circle1 = new Circle(2);

            // проверка метода ToString() для каждой ранее созданной фигуры
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine("проверка метода ToString для каждой ранее
созданной фигуры");
            Console.ForegroundColor = ConsoleColor.White;
            Console.WriteLine($"rect1: {rect1}\nsquare1:
{square1.ToString()}\ncircle1: {circle1.ToString()}");

            // проверка интерфейса IPrint на примере круга
            Console.ForegroundColor = ConsoleColor.Green;
            Console.WriteLine("\nПроверка интерфейса IPrint на примере круга");
            Console.ForegroundColor = ConsoleColor.White;
            IPrint circle2 = new Circle(4);
            circle2.Print();
        }
    }
} // Figures

```

Скриншоты работы приложения:

```

● nikod@LAPTOP-752EM865:~/bmstu/Nikonorov_Labs_2025/dotnet$ dotnet run
проверка метода ToString для каждой ранее созданной фигуры
rect1: w = 10; h = 10; area: 100
square1: side = 5; area: 25
circle1: r = 2; area: 12.57

Проверка интерфейса IPrint на примере круга
Круг: r = 4; area: 50.27

```